

Relatório de Desenvolvimento — Laboratório 1: Web Scraping

Portal: ProgramaThor (<https://programathor.com.br>) · **Área:** vagas de desenvolvimento em **Python**

Entregues: coleta_vagas.py, vagas_programathor.csv / .json, robots_programathor.txt, sitemap_programathor.txt, este relatório.

1. Decisões iniciais

O ProgramaThor foi escolhido porque as páginas são renderizadas no servidor (o HTML chega completo), a paginação é por URL (?page=N) com link *Next* explícito, e a faixa salarial é opcional — o que exercita o tratamento de campos ausentes. Catho, InfoJobs e LinkedIn foram descartados por exigirem login ou bloquearem robôs. A coleta é em duas camadas: a listagem fornece só os links; os campos vêm da página individual da vaga, mais completa que o card resumido.

2. Robots.txt e sitemap.xml

A verificação é feita pelo programa antes de qualquer coleta (`verificar_robots_e_sitemap()`): baixa `/robots.txt`, salva em disco, extrai por regex as regras `Disallow`: e as declarações `Sitemap`: e carrega tudo num `RobotFileParser`. O arquivo é curto e permissivo:

```
Sitemap: https://programathor.com.br/sitemap.xml
User-agent: *      Disallow: /admin/   /user/   /users/   /company/
```

`User-agent: *` vale para todos os robôs e os quatro bloqueios são de áreas privadas (painel administrativo, perfil de candidatos, área restrita de empresas). Não há regra `Allow` e **nenhum bloqueio incide sobre `/jobs`**, onde ficam a listagem e as vagas: `can_fetch('/jobs')` retornou `True`. O programa consulta `can_fetch()` três vezes — para `/jobs` antes de o Selenium abrir a listagem, para a URL de resultados e para cada vaga no laço principal — e aborta se a permissão for negada; se o `robots.txt` estiver inacessível por erro de rede, também aborta, pois sem ler as regras não se coleta. O bloqueio é de `/company/` (singular) e os perfis públicos usam `/companies/` (plural), mas o script não visita esses perfis: lê o nome da empresa na âncora da própria vaga. O sitemap é declarado no `robots.txt` (*Sitemap Autodiscovery*) e o script segue essa declaração em vez de adivinhar o caminho padrão, tratando também o caso de índice (`<sitemapindex>`). Boas práticas: `User-Agent` identificado com o sufixo (`projeto-academico-web-scraping`), 1,5 s entre requisições, `timeout` de 20 s e até 3 tentativas por página.

3. Ferramentas por etapa

Etapas	Ferramenta	Papel
Robots e sitemap	<code>requests</code> , <code>urllib.robotparser</code> , <code>re</code>	baixar, interpretar regras, validar permissões
Busca pelo assunto	Selenium (Chrome headless)	abrir o site, acionar o menu de skills, clicar em "Python"
Navegação nas páginas	<code>requests</code> + <code>BeautifulSoup</code>	baixar cada página de resultados e achar o link <i>Next</i>
Extração dos campos	<code>BeautifulSoup</code> + <code>re</code>	título, empresa, local, salário, requisitos, descrição
Limpeza e deduplicação	funções próprias (<code>remover_duplicatas</code> , <code>limpar</code>)	remover repetições e padronizar campos ausentes
Gravação	<code>pandas</code> / <code>csv</code> / <code>json</code>	gerar <code>.csv</code> (UTF-8 com BOM) e <code>.json</code>

A busca é feita **pelo código**: o Selenium abre `/jobs`, clica em "Todos os skills" e clica no link de texto exato Python, deixando o site conduzir à página de resultados (`driver.current_url`). Se o Selenium falhar, o *fallback* usa a URL fixa de resultados e registra no log que a execução não cumpre o requisito da busca interativa — em vez de falhar em silêncio.

4. Métodos de busca utilizados

#	Método	O que localiza
1	<code>XPath //*[contains(normalize-space(text()), 'Todos os skills')]</code>	rótulo que abre o modal de tecnologias
2	<code>XPath //a[normalize-space(text())='Python']</code>	âncora de texto exato no modal (não casa "PyTorch")
3	<code>CSS a[href*='/jobs/'] + regex ^/jobs/\d+--[^/?#]+\$</code>	links de vaga na listagem; o regex valida a URL canônica e descarta filtros e <code>/jobs-python</code>
4	<code>CSS a[rel][href] + regex [?&]page=(\d+)</code>	link da próxima página pelo atributo <code>rel</code> (próx/next) — 1º critério
5	<code>CSS a[href] + texto (next, >, próxima) + mesmo regex</code>	2ª opção para o link da próxima página
6	<code>Regex ([?&])page=\d+&?</code>	remove <code>page=N</code> para remontar a URL quando não há link <i>Next</i>
7	<code>CSS h1</code>	título da vaga
8	<code>find_all("h3")</code>	1º <code><h3></code> , que fecha o bloco de cabeçalho da vaga
9	<code>CSS a[href*='/companies/'] + regex /companies/\d+</code>	nome da empresa; o id numérico exclui <code>/companies/sign_in</code> ("Como empresa")
10	<code>find_next(["h2", "h3"])</code>	fallback da empresa: só aceita <code><h2></code> antes do 1º <code><h3></code>
11	<code>find_all(["h2", "h3", "h4", "h5"]) + regex Requisitos Qualifica, Atividades Responsabilidades Sobre a vaga, Descri[cç][aã]o da empresa</code>	títulos das seções cujo conteúdo é extraído

#	Método	O que localiza
12	CSS a[href^='/jobs-']	tags de tecnologia (exclui /jobs-city/ e rodapé)
13	Regex Localiza[cc][aã]o:\s*([^\n]+)	cidade/modalidade após "Localização:"
14	Regex Sal[aá]rio:\s*([^\n]+)	valor após "Salário:"
15	Regex R\\$[s?d{1,3}](?:\.\d{3})+(?:,\d{2})?	valor em R\$ no corpo; o milhar obrigatório evita ler "R\$ 50 milhões"
16	Regex \b(CLT PJ Est[aá]gio Freelancer)\b	tipo de contrato
17	Regex \b(J[uú]nior Pleno S[eê]nior Est[aá]gio Trainee)\b	nível (no título, depois no cabeçalho)
18	Regex Veja vagas similares N[ã]o perca nenhuma oportunidade	fim do anúncio — impede ler as vagas similares
19	Regex ^\s*Disallow:\s*(\S*) e ^\s*Sitemap:\s*(\S+)	regras e declarações de sitemap no robots.txt
20	Regex <loc>\s*(.*)\s*</loc>	cada URL no XML do sitemap
21	Regex \s+	normalização de espaços nos campos textuais

Duas armadilhas corrigidas. O método 5 testava só o *texto* da âncora, e o card de vagas com a tecnologia **NextJS** contém "next": virava "próxima página" e a 3ª página coletada era a página de uma vaga. Passou-se a exigir texto de avanço e href de paginação, com rel como 1º critério. No método 11, o conteúdo é montado percorrendo a árvore em **ordem de documento**, não pelos irmãos do título: com irmãos, a seção vazava para a seguinte quando o próximo título estava aninhado numa <div>, e vinha vazia quando o título estava dentro de um *wrapper*.

5. Tratamento dos dados

Duplicatas. Três barreiras: um set de URLs vistas durante a varredura e, em `remove_duplicatas()`, descarte por link repetido e por título + empresa repetidos — o mesmo anúncio republicado sob IDs diferentes, observado no portal. A segunda chave só vale quando a empresa foi identificada, senão vagas homônimas de empresas distintas colapsariam numa só. A função é única para os dois caminhos de gravação (com e sem pandas), que antes usavam critérios diferentes e geravam arquivos distintos.

Campos ausentes. Toda extração passa por `limpar()`, que normaliza espaços, trunca textos longos (3.000 caracteres, cortando na última palavra) e devolve "Nao informado" quando o valor é vazio. O caso mais frequente não é o campo vazio: o portal escreve Salário: Não especificado, string não vazia que era gravada como faixa válida — a estatística acusava zero vagas sem salário, quando eram quase 70%. `limpar_opcional()` converte esses rótulos ("Não especificado", "A combinar", "-") em "Nao informado"; só então a cascata funciona: rótulo "Salário:" → valor em R\$ no anúncio → "Nao informado". `normalizar_nivel()` e `normalizar_contrato()` removem acentos e caixa, evitando que Sênior, Senior e SENIOR virem categorias distintas.

Exceções. `baixar_pagina()` encapsula erros de rede, respeita o timeout, repete até 3 vezes e devolve None em vez de propagar a falha; o laço pula a vaga. `extrair_vaga()` envolve o *parsing* em try/except e o Selenium usa try/except/finally, garantindo `driver.quit()`. Uma página fora do ar não interrompe a coleta.

6. Resultados

Indicador	Valor
Regras Disallow lidas / <code>can_fetch('/jobs')</code>	4 / True
URLs no sitemap	30.382 (30.364 contendo /jobs)
Links únicos coletados nas 3 páginas	44 (15 + 15 + 14)
Páginas de vaga com HTTP 500 no portal	2 (puladas sem interromper a coleta)
Registros finais após deduplicação	42 (13 da pág. 1, 15 da pág. 2, 14 da pág. 3)
Empresas distintas / vagas sem empresa ou local	30 / 0
Vagas sem faixa salarial divulgada	29 de 42 (69%)
Nível / contrato	Senior 20 · Pleno 19 · Junior 3 / PJ 26 · CLT 16

As 42 vagas têm Python na coluna *tecnologias*, confirmando que busca e paginação ficaram no recorte pretendido. A limitação principal é a dependência do HTML — por isso preferiram-se âncoras estáveis (padrão de URL, rótulos de texto, tags semânticas) a classes CSS. A faixa salarial ausente não pode ser inferida: análises de remuneração se apoiam em 13 observações.

7. Declaração de uso de IA

Foi utilizada a ferramenta **Claude (Anthropic)**, via **Claude Code**, nas seguintes tarefas:

Tarefa	Uso
Desenvolvimento do código	apoio na escrita e refatoração das funções de coleta, extração, limpeza e gravação
Revisão de código	identificação e correção de falhas: checagem do robots.txt antes do acesso via Selenium, casamento de texto na delimitação do cabeçalho e limite no fallback do nome da empresa
Redação do relatório	organização, redação e revisão deste documento
Organização do projeto	correção da estrutura do repositório git

A escolha do portal e do recorte, a execução das coletas e a conferência dos dados gerados foram feitas pelo autor.